

Software Requirements Specification (SRS)

CCRM - Centurion Customer Relationship Management System

Version: 1.5.0
Date: June 2, 2026
Organization: CUTM Admissions
Project Manager: Principal, CUTM

1. INTRODUCTION

1.1 Purpose

This document specifies the functional and non-functional requirements for CCRM (Centurion Customer Relationship Management), a comprehensive lead and student management system for CUTM (Centurion University of Technology and Management) admissions process.

1.2 Scope

CCRM is a web-based application designed to:

- Manage 200,000+ leads efficiently with server-side pagination
- Track student journey from lead → application → enrollment → payment
- Enable counselors, managers, and admins to collaborate on student recruitment
- Automate daily reporting and backup processes
- Integrate with multiple marketing channels (Facebook, Instagram, Google Ads, LinkedIn, WhatsApp, SMS, RCS)
- Process online payments through Razorpay and PayU
- Track telephony interactions via Ameyo/Exotel

1.3 Definitions and Acronyms

Term	Definition
Lead	A prospective student interested in CUTM
Counselor	Staff member responsible for lead follow-up
Manager	Supervisor overseeing a team of counselors
Admin	System administrator with full access
CRM	Customer Relationship Management
SRS	Software Requirements Specification
SDD	Software Design Document
UTR	Unique Transaction Reference (for payments)
KPI	Key Performance Indicator
IST	Indian Standard Time (UTC+5:30)

2. OVERALL DESCRIPTION

2.1 Product Vision

A unified platform for educational institution admissions that:

- Consolidates leads from multiple marketing channels
- Enables efficient lead assignment and follow-up
- Tracks conversion through application, payment, and enrollment
- Provides real-time analytics and performance metrics
- Automates routine tasks (emails, backups, reports)
- Scales to handle 200,000+ concurrent leads

2.2 Product Features (High-Level)

2.2.1 Lead Management

- Import leads from multiple sources (Facebook Ads, Google Ads, Instagram, LinkedIn, manual upload)
- Assign leads to counselors based on campus/course
- Track lead stage: Untouched → Contacted → Interested → Process for Payment → Payment Success → Converted
- Real-time lead status visibility
- Bulk operations (assign, transfer, delete with confirmation)
- Lead search, filter, and bulk upload

2.2.2 Application Management

- Create applications from leads
- Track application stages (Draft, Submitted, Approved, Rejected, Enrolled)
- Document upload for each application
- Application verification by admins

2.2.3 Payment Management

- Online payment integration (Razorpay, PayU)
- Manual payment entry with UTR number
- Admin verification of payments
- Revenue tracking (only admin-verified payments count)
- Payment status tracking

2.2.4 User & Team Management

- Role-based access (Counselor, Manager, Admin)
- User management (create, edit, deactivate)
- Team structure with manager-to-counselor relationships
- Activity logging (who changed what, when)
- Bulk user import via Excel

2.2.5 Analytics & Reporting

- Dashboard with KPIs (Total Leads, Applications, Revenue, Enrollments)
- Campus-wise statistics
- Counselor-wise productivity report

- Daily automated email reports
- Lead stage distribution
- Top performers ranking
- Target vs. achievement tracking

2.2.6 Integration & Automation

- Multi-channel lead import (Facebook, Google Ads, Instagram, LinkedIn, WhatsApp, SMS)
- Automated email alerts to counselors
- Automated daily email reports at 3:00 AM IST
- Automated S3 backup at 3:00 AM IST
- Webhook support for lead ingestion
- RCS Business Messaging support

2.2.7 Data Management

- Database backup to AWS S3 (daily)
- Log archival
- Data export (CSV, Excel)
- Bulk operations with audit trails

3. FUNCTIONAL REQUIREMENTS

3.1 Lead Management (FR-LM)

FR-LM-1: Lead Import

- **Requirement:** System shall support importing leads from multiple sources
- **Sources:** Facebook Ads, Google Ads, Instagram Ads, LinkedIn, WhatsApp Bot, manual upload
- **Acceptance Criteria:**
 - Leads can be imported via webhook (marketing channels)
 - Leads can be uploaded via Excel file
 - Duplicate detection prevents duplicate lead entries
 - Lead deduplication based on email/mobile combination
 - Bulk import progress tracking with error reporting

FR-LM-2: Lead Assignment

- **Requirement:** Counselors and leads shall be assigned to counselors/managers
- **Acceptance Criteria:**
 - Manual assignment by managers/admins
 - Bulk assignment via dropdown selector
 - Assignment creates activity log entry
 - Assigned counselor receives email notification
 - Lead history shows all assignment changes

FR-LM-3: Lead Stages

- **Requirement:** System shall track lead progression through defined stages

- **Stages:**

1. **Untouched** — Lead received, not yet contacted
2. **Contacted** — Counselor has reached out
3. **Follow Up** — Initial contact made, awaiting response
4. **Interested** — Lead shows interest in admission
5. **Process for Payment** — Student proceeding towards payment
6. **Payment Success** — Lead stage (unverified payment)
7. **Converted** — Student enrolled

- **Acceptance Criteria:**

- Only assigned counselor can change own leads' stages
- Managers can change team leads' stages
- Admins can change any lead's stage
- Stage change creates audit log
- Timestamps recorded for each stage

FR-LM-4: Lead Search & Filter

- **Requirement:** Counselors shall search/filter leads efficiently

- **Acceptance Criteria:**

- Search by name, email, mobile, campus, course
- Filter by stage, owner (counselor), campus, date range
- Server-side pagination (50 leads per page max)
- Sort by name, date assigned, stage, last updated
- Quick filters ("My Leads", "Untouched", "Interested", etc.)

FR-LM-5: Lead Details

- **Requirement:** Detailed lead information shall be accessible

- **Acceptance Criteria:**

- Display: Name, Email, Mobile, Campus, Course, Source, Owner, Stage, Timestamps
- Edit inline: Name, Mobile (with confirmation)
- View activity history (all actions on this lead)
- View assignment history (who owned this lead when)
- View application(s) linked to this lead
- View payment(s) linked to this lead

3.2 Application Management (FR-AM)

FR-AM-1: Create Application

- **Requirement:** Counselors shall create applications from leads

- **Acceptance Criteria:**

- Create application from lead detail page
- Pre-populate with lead data (name, email, mobile, course)
- Allow editing course selection
- Generate unique Application Number (APP-XXXX-YYYY)
- Mark lead as "Process for Payment" stage

FR-AM-2: Application Stages

- **Requirement:** Track application progression
- **Stages:**
 1. **Draft** — Not yet submitted
 2. **Submitted** — Awaiting admin review
 3. **Approved** — Admin approved, fee due
 4. **Rejected** — Admin rejected, can resubmit
 5. **Enrolled** — Payment confirmed, admission complete
- **Acceptance Criteria:**
 - Counselor submits application
 - Admin reviews and approves/rejects
 - Applicant status changes reflect in dashboard
 - Status history maintained with timestamps

FR-AM-3: Document Upload

- **Requirement:** Applicants shall upload required documents
- **Acceptance Criteria:**
 - Upload from application form
 - Support file types: PDF, JPG, PNG (max 10 MB each)
 - Store in /uploads/documents/ folder
 - Organize by App Number
 - Admin can view/download all documents

FR-AM-4: Application Export

- **Requirement:** Admins shall export application data
- **Acceptance Criteria:**
 - Export as CSV/Excel with all details
 - Include document file paths
 - Filter by status, date range, campus
 - Bulk export with progress indicator

3.3 Payment Management (FR-PM)

FR-PM-1: Online Payment Gateway

- **Requirement:** System shall process online payments
- **Providers:** Razorpay, PayU
- **Acceptance Criteria:**
 - Student can initiate payment from application
 - Redirected to payment gateway
 - Webhook receives payment confirmation
 - Payment status updated automatically
 - Student receives email receipt

FR-PM-2: Payment Verification

- **Requirement:** Admin shall verify payments with UTR number
- **Acceptance Criteria:**
 - Payment requires manual verification by admin
 - Admin must enter/confirm UTR (Unique Transaction Reference)
 - Only verified payments count towards revenue
 - UTR number is unique and searchable
 - Verification creates audit log

FR-PM-3: Revenue Tracking

- **Requirement:** System shall calculate revenue accurately
- **Acceptance Criteria:**
 - Only payments with status "Approved"/"Paid" AND valid UTR count
 - Invalid UTR or unverified payments excluded from revenue
 - Revenue dashboard shows verified amount
 - Revenue breakdown by campus, course
 - Daily revenue dashboard widget

FR-PM-4: Payment History

- **Requirement:** View complete payment history
- **Acceptance Criteria:**
 - Show all payments linked to application
 - Display amount, date, status, UTR, payment method
 - Admin can filter by date, status, UTR
 - Export payment records

3.4 User & Team Management (FR-UM)

FR-UM-1: User Roles & Permissions

- **Requirement:** Role-based access control (RBAC)
- **Roles:**
 1. **Admin** — Full system access, user management, payment verification
 2. **Manager** — Oversee team, assign leads, view team reports
 3. **Counselor** — Own leads, create applications, update stages
- **Acceptance Criteria:**
 - Login with email/password
 - Session timeout after 30 minutes of inactivity
 - Token-based authentication (JWT)
 - Role verified on every API request

FR-UM-2: User Management

- **Requirement:** Admins shall manage system users
- **Acceptance Criteria:**
 - Create new user with role assignment

- Edit user (name, email, role, status)
- Deactivate/reactivate user (soft delete)
- Bulk import users via Excel
- User activity log (login history, actions)
- Password reset via email link

FR-UM-3: Team Structure

- **Requirement:** Manager-counselor relationships
- **Acceptance Criteria:**
 - Assign counselors to managers
 - Manager sees only own team leads
 - Manager can reassign team leads
 - Counselor can see only own leads (unless assigned by manager)
 - Admin sees all leads regardless of team

FR-UM-4: Activity Logging

- **Requirement:** Audit trail for all changes
- **Acceptance Criteria:**
 - Log: User, Action, Lead ID, Timestamp, Old Value, New Value
 - Searchable by user, date range, action type
 - Admin-only access to activity logs
 - 90-day retention of logs

3.5 Analytics & Reporting (FR-AR)

FR-AR-1: Dashboard

- **Requirement:** Real-time KPI dashboard
- **Acceptance Criteria:**
 - Total Leads (count of all leads)
 - Total Applications (count of applications)
 - Revenue Collected (sum of verified payments with UTR)
 - Enrolments (count of enrolled students)
 - Campus-wise breakdown
 - Load in <2 seconds for admin, <3 seconds for counselor (role-scoped)

FR-AR-2: Productivity Report

- **Requirement:** Counselor-wise performance metrics
- **Acceptance Criteria:**
 - Show per-counselor stats:
 - Leads Assigned
 - Untouched
 - Interested
 - Process for Pay
 - App Assigned

- Pay Approved (verified payments)
- Sortable by any column
- Exportable as CSV
- Pagination (6 rows per page)
- Shows total row at bottom

FR-AR-3: Daily Email Reports

- **Requirement:** Automated email reports at 3:00 AM IST
- **Acceptance Criteria:**
 - Email contains KPI summary + counselor-wise table
 - HTML formatted with styling
 - Recipients configurable in Settings
 - Only sent if recipients list not empty
 - Requires SMTP to be configured
 - Log success/failure in system logs

FR-AR-4: Lead Stage Distribution

- **Requirement:** Analytics on lead distribution across stages
- **Acceptance Criteria:**
 - Show count of leads in each stage
 - Pie chart visualization
 - Filter by campus, date range
 - Export as report

FR-AR-5: Target vs Achievement

- **Requirement:** Admission targets and achievement tracking
- **Acceptance Criteria:**
 - Admins set monthly targets (leads, applications, enrollments)
 - Dashboard shows target vs. achieved
 - Progress bar visualization
 - Campus-wise breakdown

3.6 Integration & Automation (FR-IA)

FR-IA-1: Multi-Channel Lead Import

- **Requirement:** Support multiple lead sources
- **Sources:**
 - Facebook Lead Ads (via Graph API)
 - Instagram Lead Ads (via Meta API)
 - Google Ads Lead Forms (via webhook)
 - LinkedIn Lead Gen Forms (via webhook)
 - WhatsApp Chatbot (via WABA)
 - Manual bulk upload (Excel)
- **Acceptance Criteria:**

- Webhook endpoints for each source
- Automatic duplicate detection
- Lead data normalized and stored
- Error handling with retry logic

FR-IA-2: Email Alerts

- **Requirement:** Automated email notifications to counselors
- **Triggers:**
 - Lead assigned to counselor
 - Lead needs follow-up (via scheduled task)
 - Payment received notification
 - Application status change
- **Acceptance Criteria:**
 - Email sent within 5 minutes of trigger
 - Email contains relevant details
 - Counselor can unsubscribe from specific alerts
 - Requires SMTP configuration

FR-IA-3: Daily Backup

- **Requirement:** Automated daily backup at 3:00 AM IST
- **Backup Components:**
 - Full PostgreSQL database dump (gzipped)
 - /uploads/ directory (tarred and gzipped)
 - Server logs (last 24 hours, gzipped)
- **Storage:** AWS S3 in date-stamped folders (backups/YYYY-MM-DD/)
- **Acceptance Criteria:**
 - Backup completes within 30 minutes
 - All 3 components uploaded successfully
 - Failure logged with error details
 - S3 credentials securely stored in integration_settings

FR-IA-4: Scheduled Tasks

- **Requirement:** Automated recurring tasks
- **Tasks:**
 - Daily email report (3:00 AM IST)
 - Daily S3 backup (3:00 AM IST)
 - Follow-up reminders (configurable)
- **Acceptance Criteria:**
 - Cron jobs run reliably
 - Failures trigger admin alerts
 - Timezone: Asia/Kolkata (IST)
 - Logs all executions

3.7 Data Management (FR-DM)

FR-DM-1: Data Export

- **Requirement:** Users shall export data for external use
- **Formats:** CSV, Excel (.xlsx)
- **Acceptance Criteria:**
 - Export leads with all fields
 - Export applications with documents
 - Export payments with UTR
 - Filter before export
 - Large exports show progress (>1000 rows)

FR-DM-2: Bulk Operations

- **Requirement:** Perform actions on multiple records
- **Operations:** Assign, reassign, delete, stage change
- **Acceptance Criteria:**
 - Select multiple records (checkbox)
 - Confirm before deletion
 - Progress indicator for large operations
 - Audit log for each changed record
 - Rollback option (undo) within 5 minutes

FR-DM-3: Data Cleanup

- **Requirement:** Remove dummy/junk data
 - **Acceptance Criteria:**
 - Admin can mark records as "junk"
 - Junk records hidden from normal views
 - Separate admin view for junk records
 - Permanent delete (soft delete to archive table) only after 30 days
-

4. NON-FUNCTIONAL REQUIREMENTS

4.1 Performance (NFR-PE)

NFR-PE-1: Response Time

- Dashboard load: <2 seconds (admin), <3 seconds (counselor)
- Lead list pagination: <1 second per page
- Search/filter: <2 seconds
- API endpoints: <500ms p95 latency

NFR-PE-2: Scalability

- Support 200,000+ leads in database
- Concurrent users: 100+ simultaneous

- Server-side pagination to prevent browser freeze
- Only essential fields in list views (not lead_details JSONB)

NFR-PE-3: Throughput

- Support 1,000+ lead imports per day
- Support 100+ concurrent API calls
- Batch export of 50,000+ records

4.2 Reliability (NFR-RE)

NFR-RE-1: Availability

- 99.5% uptime SLA
- Planned maintenance windows announced 48h in advance
- Graceful degradation if services unavailable

NFR-RE-2: Data Integrity

- Transactional consistency (ACID)
- No data loss on crashes
- Automated backups every 24 hours
- Backup tested monthly

NFR-RE-3: Error Handling

- Graceful error messages to users
- Errors logged with stack traces
- Admin notification on critical errors
- Automatic retry for transient failures

4.3 Security (NFR-SE)

NFR-SE-1: Authentication

- Email/password login
- Session timeout: 30 minutes of inactivity
- JWT token-based for API
- Password minimum 8 characters

NFR-SE-2: Authorization

- Role-based access control (RBAC)
- Row-level security (counselor sees only own leads)
- API calls verify role on every request
- Admin-only endpoints require explicit admin role

NFR-SE-3: Data Protection

- HTTPS/TLS for all traffic
- Passwords hashed with bcrypt
- Sensitive fields masked (API keys, secrets)

- PII (email, mobile) encrypted at rest (optional: phase 2)

NFR-SE-4: Credential Management

- Integration credentials stored in integration_settings table
- Credentials encrypted before storage
- Never logged or exposed in error messages
- Admin can rotate credentials anytime

4.4 Usability (NFR-US)

NFR-US-1: User Interface

- Responsive design (mobile, tablet, desktop)
- Intuitive navigation
- Consistent color scheme and typography
- Dark mode support (future)

NFR-US-2: Accessibility

- WCAG 2.1 Level AA compliance
- Keyboard navigation support
- Screen reader compatible
- Sufficient color contrast

NFR-US-3: Help & Support

- In-app help documentation
- Tooltip explanations for complex fields
- Error messages in plain language
- Admin knowledge base

4.5 Maintainability (NFR-MA)

NFR-MA-1: Code Quality

- Clean, readable, well-commented code
- No magic numbers (use named constants)
- Modular architecture
- DRY (Don't Repeat Yourself) principle

NFR-MA-2: Documentation

- API documentation (endpoints, parameters, responses)
- Database schema documentation
- Deployment guide
- Troubleshooting guide

NFR-MA-3: Logging

- Structured logs (JSON format)
- Log levels: DEBUG, INFO, WARN, ERROR

- Centralized log storage
- 90-day log retention

4.6 Compatibility (NFR-CO)

NFR-CO-1: Browser Support

- Chrome (latest)
- Firefox (latest)
- Safari (latest)
- Edge (latest)

NFR-CO-2: Device Support

- Desktop (1920x1080+)
- Tablet (iPad, 768x1024+)
- Mobile (iPhone 12+, Android 11+)

NFR-CO-3: Database

- PostgreSQL 12+
 - Backup compatible with AWS RDS
-

5. SYSTEM CONSTRAINTS

5.1 Technical Constraints

- **Backend:** Node.js/Express
- **Frontend:** React 18+ with Vite
- **Database:** PostgreSQL 12+
- **Hosting:** Linux server (Ubuntu 20.04+)
- **File Storage:** Local filesystem + AWS S3

5.2 Business Constraints

- **Go-Live:** By December 2026
- **Budget:** Within allocation
- **Team Size:** 1 developer + 1 QA

5.3 Regulatory Constraints

- GDPR compliance for EU data
 - India data residency for Indian leads
 - Password complexity per government standards
 - Audit trail for sensitive operations
-

6. ACCEPTANCE CRITERIA & TESTING

6.1 Functional Testing

- All FR requirements tested with positive and negative test cases
- Test coverage: >80%
- Regression testing after each change

6.2 Performance Testing

- Load test with 100 concurrent users
- Verify response times meet NFR-PE
- Database query optimization

6.3 Security Testing

- SQL injection tests
- XSS (Cross-Site Scripting) tests
- CSRF (Cross-Site Request Forgery) tests
- Penetration testing

6.4 User Acceptance Testing (UAT)

- Real users test end-to-end workflows
- Feedback collected and prioritized
- Defects logged and tracked

7. GLOSSARY

Term	Definition
Lead	Prospective student contact
Conversion	Lead becomes enrolled student
UTR	Payment transaction reference number
KPI	Measurable performance indicator
Role	Job function (Admin, Manager, Counselor)
Stage	Position in lead lifecycle
Campus	Physical location (Bhubaneswar, etc.)
Course	Educational program (BTech, MBA, etc.)
Webhook	HTTP callback for real-time events

8. DOCUMENT CHANGE HISTORY

Version	Date	Author	Changes
1.0	Jan 2026	Team	Initial draft
1.1	Feb 2026	Team	Added payment verification requirements
1.2	Mar 2026	Team	Added automated reporting requirements
1.3	Apr 2026	Team	Refined lead stage definitions
1.4	May 2026	Team	Added S3 backup requirements
1.5	Jun 2026	Team	Final version with all requirements

End of SRS Document

Software Design Document (SDD)

CCRM - Centurion Customer Relationship Management System

Version: 1.5.0

Date: June 2, 2026

Architect: Development Team

Status: Final

1. DESIGN OVERVIEW

1.1 Architectural Style

Three-Tier Architecture:

- Presentation Layer** — React frontend (Vite)
- Application Layer** — Node.js/Express backend
- Data Layer** — PostgreSQL database + AWS S3

1.2 Key Design Principles

- Scalability:** Server-side pagination, role-scoped queries
- Security:** RBAC, JWT tokens, input validation
- Maintainability:** Modular code, clear separation of concerns
- Performance:** Indexed database queries, optimized API responses
- Reliability:** Error handling, transaction consistency, automated backups

2. SYSTEM ARCHITECTURE

2.1 High-Level Architecture Diagram

USERS (Browser)
Admin | Manager | Counselor | Lead (Public Portal)

HTTPS

FRONTEND (React 18 + Vite)

- Pages: Dashboard, LeadManager, Apps, Payments, etc
- Components: Cards, Tables, Forms, Modals
- Context: CcrmContext (state management)
- Utils: API client, auth, formatting

REST API (JSON)

BACKEND (Node.js + Express)

- Routes: /api/leads, /api/applications, /api/payments
- Auth: JWT token validation, RBAC middleware
- Services: Lead mgmt, Application mgmt, Payments
- Integrations: WhatsApp, SMS, Email, Payment gateways
- Cron Jobs: Daily email reports, S3 backups
- Error Handling: Input validation, transaction mgmt

TCP/5432

DATABASE (PostgreSQL)

- Tables: leads, applications, payments, users, etc
- Indexes: email, mobile, stage, owner, date
- Relationships: Foreign keys, constraints
- Triggers: Audit logging, timestamp updates

STORAGE & EXTERNAL SERVICES

- Local Filesystem: /server/uploads/
- AWS S3: Daily backups (backups/YYYY-MM-DD/)
- Email: SMTP (Gmail, custom providers)
- SMS/WhatsApp/RCS: Third-party gateways
- Payments: Razorpay, PayU webhooks
- Telephony: Ameyo, Exotel APIs

2.2 Technology Stack

Layer	Technology	Version	Purpose
Frontend	React	18+	UI framework
	Vite	5.4+	Build tool
	Lucide Icons	-	UI icons
	TailwindCSS	3.3+	Styling
Backend	Node.js	18+	Runtime
	Express	4.18+	Web framework
	PostgreSQL	12+	Database
	node-cron	3.0+	Job scheduling
	Nodemailer	6.10+	Email sending
	AWS SDK	3.600+	S3 operations
	Multer	1.4+	File uploads
	JWT	9.0+	Authentication
	bcryptjs	2.4+	Password hashing
Infrastructure	Ubuntu	20.04+	OS
	systemd	-	Service management
	AWS S3	-	Backup storage

3. DATABASE DESIGN

3.1 Entity-Relationship Diagram

users

- id (PK)
- email (UNIQUE)
- password (hashed)
- name
- role (Admin|Manager|Counselor|Lead)
- status (Active|Inactive)
- campus
- created_at
- updated_at

leads

- id (PK)
- email (INDEX)
- mobile (INDEX)
- name
- campus
- course
- stage (INDEX) — Untouched|Contacted|...|Converted
- owner (FK→users, INDEX) — counselor assigned
- source — Facebook|Google|Instagram|LinkedIn|Manual
- lead_details (JSONB) — custom fields, campaign data
- created_at (INDEX)
- updated_at

└─ deleted_at (soft delete)

applications

└─ id (PK)
└─ app_no (UNIQUE) — APP-XXXX-YYYY
└─ lead_id (FK→leads, INDEX)
└─ owner (FK→users) — counselor who created
└─ stage — Draft|Submitted|Approved|Rejected|Enrolled
└─ campus
└─ course
└─ created_at
└─ updated_at

payments

└─ id (PK)
└─ app_no (FK→applications, INDEX)
└─ lead_id (FK→leads, INDEX)
└─ amount (in paise, e.g., 50000 = ₹500)
└─ status — Pending|Approved|Paid|Failed|Refunded
└─ utr_number (UNIQUE, INDEX) — Unique Transaction Reference
└─ payment_method — Online|Manual
└─ gateway — Razorpay|PayU|Manual
└─ verified_by (FK→users) — admin who verified
└─ verified_at
└─ created_at
└─ updated_at

documents

└─ id (PK)
└─ app_id (FK→applications, INDEX)
└─ file_name
└─ file_path — /uploads/documents/APP-XXXX/filename
└─ file_size
└─ file_type — PDF|JPG|PNG
└─ uploaded_by (FK→users)
└─ created_at
└─ updated_at

integration_settings

└─ id (PK)
└─ key (UNIQUE, INDEX) — smtp_host, aws_access_key_id, etc.
└─ value (encrypted) — secret values
└─ updated_at
└─ created_at

activity_logs

└─ id (PK)
└─ user_id (FK→users, INDEX)
└─ entity_type — Lead|Application|Payment
└─ entity_id (INDEX)
└─ action — Create|Update|Delete|Assign
└─ old_value (JSONB)

- |— new_value (JSONB)
- |— created_at (INDEX)
- |— metadata (JSONB)

targets

- |— id (PK)
- |— month
- |— year
- |— campus
- |— target_leads
- |— target_applications
- |— target_enrollments
- |— created_at

3.2 Database Indexes

```
CREATE INDEX idx_leads_email ON leads(email);  
CREATE INDEX idx_leads_mobile ON leads(mobile);  
CREATE INDEX idx_leads_owner ON leads(owner);  
CREATE INDEX idx_leads_stage ON leads(stage);  
CREATE INDEX idx_leads_created_at ON leads(created_at DESC);  
  
CREATE INDEX idx_applications_app_no ON applications(app_no);  
CREATE INDEX idx_applications_lead_id ON applications(lead_id);  
CREATE INDEX idx_applications_owner ON applications(owner);  
  
CREATE INDEX idx_payments_app_no ON payments(app_no);  
CREATE INDEX idx_payments_lead_id ON payments(lead_id);  
CREATE INDEX idx_payments_utr_number ON payments(utr_number);  
CREATE INDEX idx_payments_status ON payments(status);  
  
CREATE INDEX idx_activity_logs_user_id ON activity_logs(user_id);  
CREATE INDEX idx_activity_logs_entity ON activity_logs(entity_type, entity_id);  
CREATE INDEX idx_activity_logs_created_at ON activity_logs(created_at DESC);  
  
CREATE INDEX idx_integration_settings_key ON integration_settings(key);
```

3.3 Key Constraints & Relationships

```

-- Foreign Keys
ALTER TABLE leads
ADD CONSTRAINT fk_leads_owner FOREIGN KEY (owner) REFERENCES users(name);

ALTER TABLE applications
ADD CONSTRAINT fk_applications_lead FOREIGN KEY (lead_id) REFERENCES leads(id) ON DELETE
CASCADE;

ALTER TABLE payments
ADD CONSTRAINT fk_payments_app FOREIGN KEY (app_no) REFERENCES applications(app_no);

-- Unique Constraints
ALTER TABLE users ADD CONSTRAINT uq_email UNIQUE (email);
ALTER TABLE applications ADD CONSTRAINT uq_app_no UNIQUE (app_no);
ALTER TABLE payments ADD CONSTRAINT uq utr UNIQUE (utr_number);

-- Check Constraints
ALTER TABLE leads
ADD CONSTRAINT chk_stage CHECK (stage IN ('Untouched', 'Contacted', 'Follow Up', 'Interested',
'Process for Payment', 'Payment Success', 'Converted'));

```

4. API DESIGN

4.1 API Architecture

Base URL: <https://crm.cutmap.ac.in/api>

Authentication: JWT Bearer token in Authorization header

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

4.2 API Endpoints (Core)

Authentication

POST	/auth/login	— Login with email/password
POST	/auth/logout	— Logout and invalidate token
POST	/auth/refresh	— Refresh JWT token
POST	/auth/reset-password	— Password reset via email

Leads

GET	/leads	— List leads (paginated, role-scoped)
GET	/leads/:id	— Get single lead details
POST	/leads	— Create lead (admin/manager)
PUT	/leads/:id	— Update lead
DELETE	/leads/:id	— Soft delete lead
PUT	/leads/:id/stage	— Update lead stage
PUT	/leads/:id/owner	— Assign lead to counselor
POST	/leads/bulk/upload	— Bulk import leads (Excel)
POST	/leads/bulk/assign	— Bulk assign leads

Applications

GET	/applications	— List applications (paginated)
GET	/applications/:id	— Get single application
POST	/applications	— Create application
PUT	/applications/:id	— Update application
PUT	/applications/:id/stage	— Update application stage
GET	/applications/:app_no/documents	— List documents
POST	/applications/:app_no/documents	— Upload document
DELETE	/applications/:app_no/documents/:id	— Delete document

Payments

GET	/payments	— List payments
GET	/payments/:id	— Get single payment
POST	/payments	— Record payment
PUT	/payments/:id	— Update payment
POST	/payments/:id/verify	— Admin verify with UTR
POST	/payments/webhook/razorpay	— Razorpay webhook
POST	/payments/webhook/payu	— PayU webhook

Users & Teams

GET	/users	— List users (admin only)
POST	/users	— Create user (admin only)
PUT	/users/:id	— Edit user (admin only)
DELETE	/users/:id	— Deactivate user (admin only)
GET	/users/:id/activity	— User's recent actions
POST	/users/bulk/import	— Bulk import users (Excel)
GET	/teams/:manager_id	— List team members

Analytics & Reports

GET	/dashboard/stats	— KPI summary (role-scoped)
GET	/campus/stats	— Campus-wise statistics
GET	/users/:user_id/stats	— Individual counselor stats
GET	/targets/achievement	— Month-wise achievement
POST	/targets	— Set admission targets
GET	/activity-logs	— Audit trail (admin only)

Integrations

GET	/integration-settings	— List all configured integrations
POST	/integrations	— Save integration settings
POST	/admin/test-daily-report	— Trigger email + backup (admin)

4.3 Request/Response Format

Request:

```
{
  "method": "GET",
  "url": "https://crm.cutmap.ac.in/api/leads?page=1&limit=50&owner=John%20Doe",
  "headers": {
    "Authorization": "Bearer <token>",
    "Content-Type": "application/json"
  }
}
```

Response (Success):

```
{
  "status": "success",
  "data": {
    "leads": [
      {
        "id": 1,
        "name": "Rahul Kumar",
        "email": "rahul@gmail.com",
        "mobile": "9876543210",
        "stage": "Interested",
        "owner": "John Doe",
        "created_at": "2026-06-01T10:30:00Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 50,
      "total": 203978,
      "total_pages": 4080
    }
  }
}
```

Response (Error):

```
{
  "status": "error",
  "error": "Unauthorized",
  "code": 401,
  "message": "Invalid or expired token"
}
```

4.4 Pagination Strategy

Server-Side Pagination:

- Default limit: 50 records per request
- Maximum limit: 500 records per request
- Offset-based: ?page=1&limit=50 translates to OFFSET 0 LIMIT 50
- Prevents loading 200k+ records into browser memory

SQL Query Pattern:

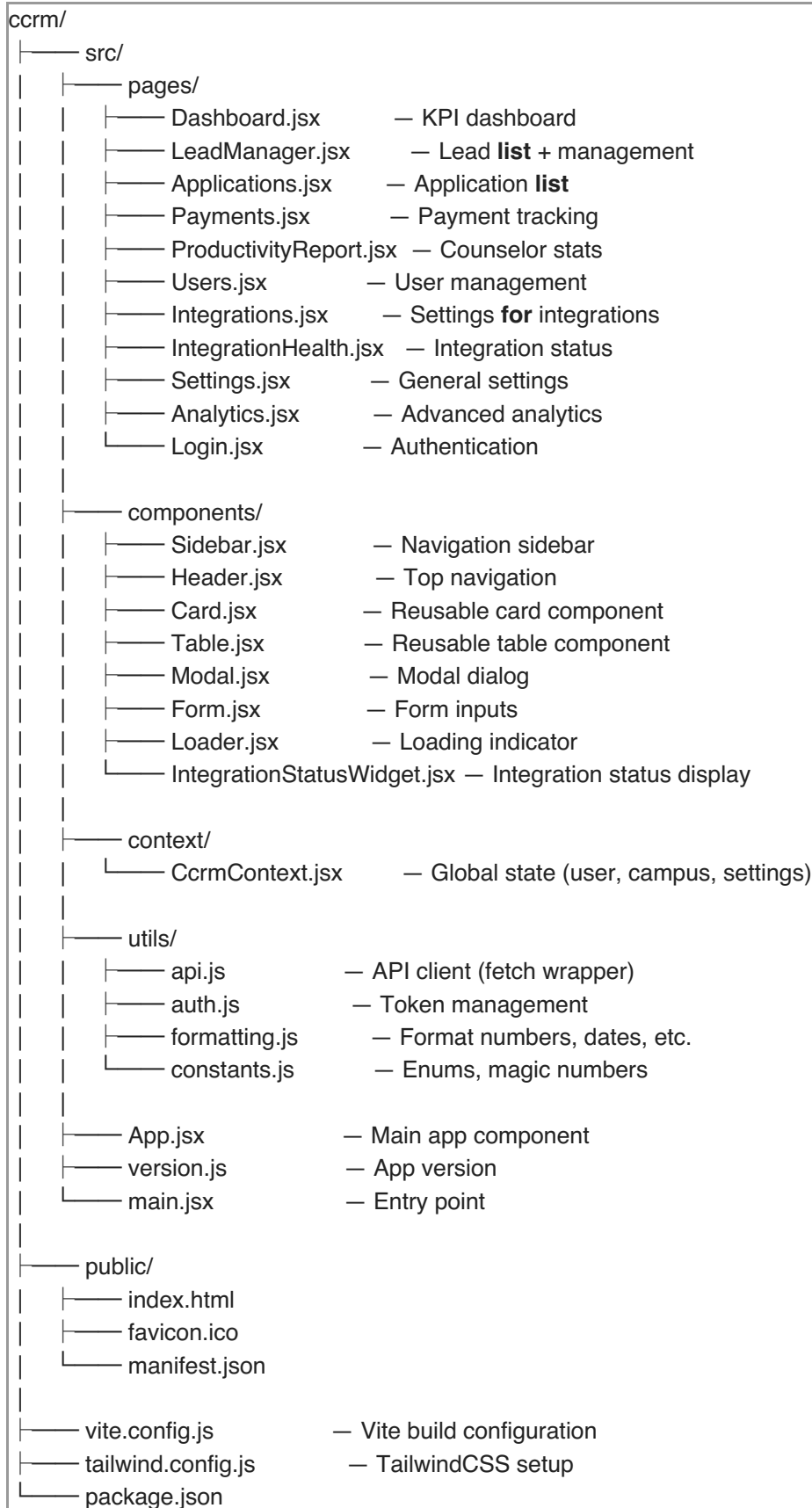
```
SELECT id, name, email, mobile, stage, owner, created_at
FROM leads
WHERE owner = $1 AND deleted_at IS NULL
ORDER BY created_at DESC
LIMIT 50 OFFSET 0;
```

4.5 Error Handling & Status Codes

Code	Scenario
200	OK — Request succeeded
201	Created — Resource created
400	Bad Request — Invalid input
401	Unauthorized — Missing/invalid token
403	Forbidden — Insufficient permissions
404	Not Found — Resource doesn't exist
409	Conflict — Duplicate entry
500	Server Error — Internal error
503	Service Unavailable — Maintenance

5. FRONTEND ARCHITECTURE

5.1 Project Structure



5.2 Component Architecture

Page Components:

- Fetch data from API on mount

- Manage page-level state
- Handle user interactions
- Emit actions to context when needed

Reusable Components:

- Pure functional components (no side effects)
- Accept props for data and callbacks
- No API calls (handled by pages)
- Focused on presentation

Context Usage:

- Global state: currentUser, activeCampus, showToast
- Auth token storage
- User profile and permissions
- Toast notifications

5.3 State Management

```
// CcrmContext provides:
{
  // Auth & User
  currentUser: { id, email, name, role },
  isLoggedIn: boolean,
  logout: () => void,

  // UI State
  activeCampus: string,
  setActiveCampus: (campus) => void,
  counselors: array,
  managers: array,

  // Notifications
  showToast: (message, type) => void,

  // Actions
  saveTarget: (targetForm) => Promise,
  refreshData: () => Promise
}
```

5.4 Key Pages & Flows

Dashboard:

- Displays KPIs (Total Leads, Applications, Revenue, Enrollments)
- Campus filter buttons
- Target vs. achievement tracker
- Load time: <3 seconds
- Role-scoped: Counselor sees only own stats, Manager sees team stats, Admin sees all

Lead Manager:

- Paginated lead list (50 per page)

- Search/filter by name, email, mobile, stage, owner
- Inline name editing with pencil icon
- Bulk operations (assign, delete)
- Lead count respects role scoping

Productivity Report:

- Counselor-wise statistics table
- Quick Summary view (Lead Assigned, Untouched, Interested, etc.)
- Exportable as CSV
- Pagination (6 rows per page)

6. BACKEND ARCHITECTURE

6.1 Server Structure

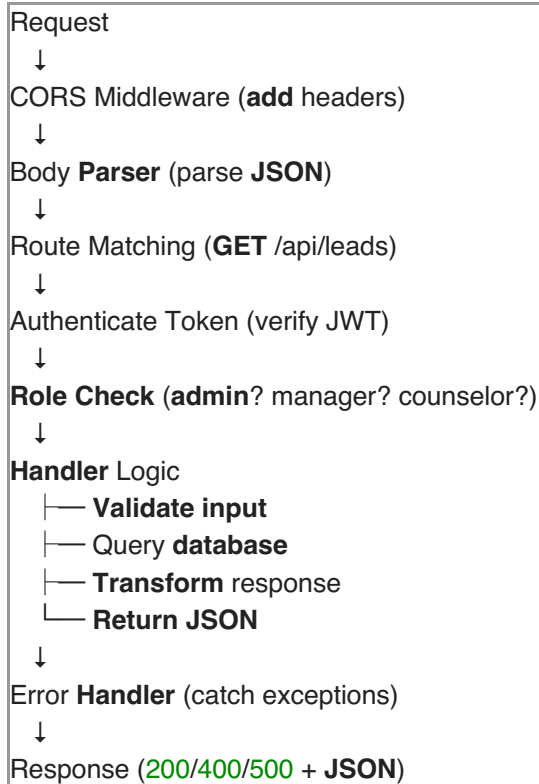
```
server/
├── index.js           — Main server file
├── db.js             — Database connection pool
├── package.json      — Dependencies
├── .env              — Environment variables
├── uploads/          — Uploaded files
│   ├── documents/    — Application documents
│   └── avatars/      — User avatars
├── logs/
│   └── server.log    — Application logs
```

6.2 Middleware Stack

```
// Global Middleware (applied to all requests)
app.use(cors())           // CORS headers
app.use(express.json())   // JSON parsing
app.use(express.static(distPath)) // Static frontend
app.use(errorHandler)     // Error handling

// Route-level Middleware
app.get('/api/...', authenticateToken, handler) // Auth required
app.post('/api/...', authenticateToken, handler)
app.put('/api/...', authenticateToken, handler)
app.delete('/api/...', authenticateToken, roleCheck('Admin'), handler) // Admin only
```

6.3 Request Flow



6.4 Key Functions

Authentication

```
// Generate JWT on login
const token = jwt.sign({ id, email, role }, SECRET, { expiresIn: '24h' })

// Verify token on each request
app.use((req, res, next) => {
  const token = req.headers['authorization']?.split(' ')[1]
  try {
    req.user = jwt.verify(token, SECRET)
    next()
  } catch (e) {
    res.status(401).json({ error: 'Unauthorized' })
  }
})
```

Role Scoping

```
// Counselor sees only own leads
if (req.user.role === 'Counselor') {
  query += ` AND owner = $1 `
  params.push(req.user.name)
}

// Manager sees team leads
if (req.user.role === 'Manager') {
  const team = await getTeamMembers(req.user.name)
  query += ` AND owner IN (${team.map((_, i) => `${i+1}`).join(',')}) `
  params.push(...team)
}

// Admin sees all leads (no WHERE clause)
```

Pagination

```
const page = Math.max(1, parseInt(req.query.page) || 1)
const limit = Math.min(500, parseInt(req.query.limit) || 50)
const offset = (page - 1) * limit

const result = await pool.query(
  `SELECT * FROM leads WHERE ... LIMIT $1 OFFSET $2`,
  [limit, offset]
)

res.json({
  data: result.rows,
  pagination: { page, limit, total: result.rowCount }
})
```

Daily Email Report (Cron)

```
async function sendProductivityEmailReport() {
  const stats = await getStatsFromDB()
  const recipients = await getIntegrationSetting('report_email_recipients')
  const html = buildHtmlTable(stats)

  const transporter = nodemailer.createTransport(smtpConfig)
  await transporter.sendMail({
    from: config.from,
    to: recipients,
    subject: `Productivity Report — ${new Date().toLocaleDateString()}`,
    html: html
  })
}

cron.schedule('0 3 * * *', sendProductivityEmailReport, {
  timezone: 'Asia/Kolkata'
})
```

S3 Backup (Cron)

```
async function performS3Backup() {
  const s3 = new S3Client({ credentials, region })
  const dateStr = new Date().toISOString().split('T')[0]

  // Database dump
  const db = await execAsync(`pg_dump ... | gzip`)
  await s3.send(new PutObjectCommand({
    Bucket: bucket,
    Key: `backups/${dateStr}/db.sql.gz`,
    Body: db
  }))

  // Uploads tar
  const uploads = await execAsync(`tar -czf - uploads/`)
  await s3.send(new PutObjectCommand({
    Bucket: bucket,
    Key: `backups/${dateStr}/uploads.tar.gz`,
    Body: uploads
  }))

  // Logs
  const logs = await execAsync(`journalctl ... | gzip`)
  await s3.send(new PutObjectCommand({
    Bucket: bucket,
    Key: `backups/${dateStr}/server.log.gz`,
    Body: logs
  }))
}
```

7. SECURITY DESIGN

7.1 Authentication & Authorization

Login

↓
Email + **Password** → Verify against bcrypt hash
↓
Generate JWT token (24h expiry)
↓
Store **in** localStorage **on** frontend
↓
Each API request includes token **in header**
↓
Server verifies JWT signature
↓
Extract **user role from** token
↓
Check role against endpoint requirements
↓
Allow/deny request

7.2 Password Security

```
// On registration/password change:  
const hashedPassword = await bcrypt.hash(plainPassword, 10)  
// Store hashedPassword in database (never plain text)  
  
// On login:  
const isValid = await bcrypt.compare(plainPassword, hashedPassword)
```

7.3 Data Protection

```
// Sensitive fields are masked in API responses  
const response = {  
  id: 1,  
  email: 'user@example.com',  
  // secret: '*****' (not sent)  
  // password: '*****' (not sent)  
}  
  
// Credentials stored in database, encrypted:  
const encrypted = encrypt(value, KEY)  
// Decrypt only when needed
```

7.4 Input Validation

```
// Validate all user inputs before using
const email = req.body.email?.trim()
if (!email || !/^[^s@]+@[^s@]+\.[^s@]+$/.test(email)) {
  return res.status(400).json({ error: 'Invalid email' })
}

// Parameterized queries prevent SQL injection:
const result = await pool.query(
  'SELECT * FROM users WHERE email = $1',
  [email] // Parameterized, not string concatenation
)
```

8. PERFORMANCE OPTIMIZATION

8.1 Database Optimization

Indexes:

- Email, mobile (lead lookup)
- Owner, stage (filtering)
- created_at (sorting)
- utr_number (payment lookup)

Query Optimization:

- Use EXPLAIN ANALYZE to identify slow queries
- Avoid N+1 queries (use JOINS)
- Limit SELECT to needed columns (not SELECT *)
- Use pagination (LIMIT/OFFSET)

Connection Pooling:

```
const pool = new Pool({
  connectionString: DATABASE_URL,
  max: 20, // Max connections
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
})
```

8.2 API Optimization

Response Caching:

- Cache integration settings (5 min TTL)
- Cache user roles (session duration)
- Don't cache: real-time lead counts, payments

Payload Optimization:

- Remove lead_details JSONB from list views (72MB → 2MB)
- Only send needed fields
- Compress JSON responses (gzip)

Rate Limiting:

```
const rateLimit = require('express-rate-limit')
app.use(rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100 // max 100 requests per IP
}))
```

8.3 Frontend Optimization

Code Splitting:

- Lazy load pages using React.lazy()
- Load integrations only when opened

Caching:

- Cache static assets (JS, CSS) with hash in filename
- Browser caches for 1 year
- Cache-busting on deploy

Rendering:

- Use React.memo() for expensive components
 - Avoid unnecessary re-renders
 - Use key prop in lists
-

9. TESTING STRATEGY

9.1 Unit Testing

```
// Test individual functions
describe('passwordValidation', () => {
  it('should require minimum 8 characters', () => {
    expect(isValidPassword('Pass123')).toBe(false)
    expect(isValidPassword('Password123')).toBe(true)
  })
})
```

9.2 Integration Testing

```
// Test API endpoints
describe('POST /api/leads', () => {
  it('should create a new lead', async () => {
    const res = await request(app)
      .post('/api/leads')
      .set('Authorization', `Bearer ${token}`)
      .send({ name: 'Rahul', email: 'rahul@example.com' })

    expect(res.status).toBe(201)
    expect(res.body.data.id).toBeDefined()
  })
})
```

9.3 Performance Testing

```
# Load test with 100 concurrent users
ab -n 10000 -c 100 https://crm.cutmap.ac.in/api/leads?page=1

# Expected: <500ms p95 latency, <2% error rate
```

9.4 Security Testing

- SQL Injection: Try email: ''; DROP TABLE users; --"
- XSS: Try name: "<script>alert('xss')</script>"
- CSRF: Verify same-site cookie flag
- Penetration testing by external firm

10. DEPLOYMENT ARCHITECTURE

10.1 Deployment Environment

```
Production Server (Linux)
├── Node.js process (ccrm-backend systemd service)
├── PostgreSQL database
├── /var/www/ccrm/
│   ├── server/ (backend code)
│   ├── ccrm/ (frontend build)
│   └── uploads/ (uploaded files)
├── Reverse proxy (Nginx)
└── HTTPS termination
```

10.2 Deployment Process

1. Developer commits **to** GitHub
2. CI/CD pipeline (**if** enabled):
 - Run tests
 - Build frontend
 - Deploy **to** production
3. **Manual** deployment (current):
 - Run DEPLOY.sh on **server**
 - Copy files **to** /var/www/ccrm/
 - Restart ccrm-backend **service**

10.3 Monitoring & Logging

Application Logs:

```
sudo journalctl -u ccrm-backend -f
```

Key Metrics:

- Response time (p50, p95, p99)
 - Error rate
 - Database query time
 - Cron job execution status
-

11. SCALABILITY CONSIDERATIONS

11.1 Current Architecture

- **Single Server:** Node.js + PostgreSQL on one machine
- **Leads:** 200,000+
- **Concurrent Users:** 100+
- **Storage:** Local filesystem + S3

11.2 Future Scalability (Phase 2)

Horizontal Scaling:

- Load balancer (Nginx, HAProxy)
- Multiple Node.js instances
- Connection pooling to PostgreSQL

Database Scaling:

- Read replicas for analytics
- Sharding by campus (if >1M leads)
- Archive old leads to cold storage

Caching Layer:

- Redis for session storage
 - Cache integration settings
 - Cache frequently accessed data
-

12. DISASTER RECOVERY

12.1 Backup Strategy

Daily Backups (3:00 AM IST):

- Database: Full PostgreSQL dump (gzipped)
- Files: uploads/ directory (tarred)
- Logs: Last 24 hours

Storage: AWS S3 (replicated across regions)

Retention: 30 days minimum (configurable via S3 lifecycle policies)

12.2 Recovery Procedures

Database Recovery:

```
aws s3 cp s3://bucket/backups/2026-06-02/db.sql.gz .
gunzip db.sql.gz
psql -U ccrm_user ccrm_db < db.sql
```

Files Recovery:

```
aws s3 cp s3://bucket/backups/2026-06-02/uploads.tar.gz .
tar -xzf uploads.tar.gz -C /var/www/ccrm/
```

RTO (Recovery Time Objective): <4 hours

RPO (Recovery Point Objective): <24 hours

13. DOCUMENT CHANGE HISTORY

Version	Date	Author	Changes
1.0	Jan 2026	Team	Initial design
1.1	Feb 2026	Team	Added payment verification
1.2	Mar 2026	Team	Optimized database queries
1.3	Apr 2026	Team	Added cron job architecture
1.4	May 2026	Team	Refined API design
1.5	Jun 2026	Team	Final version with S3 backup

End of SDD Document